



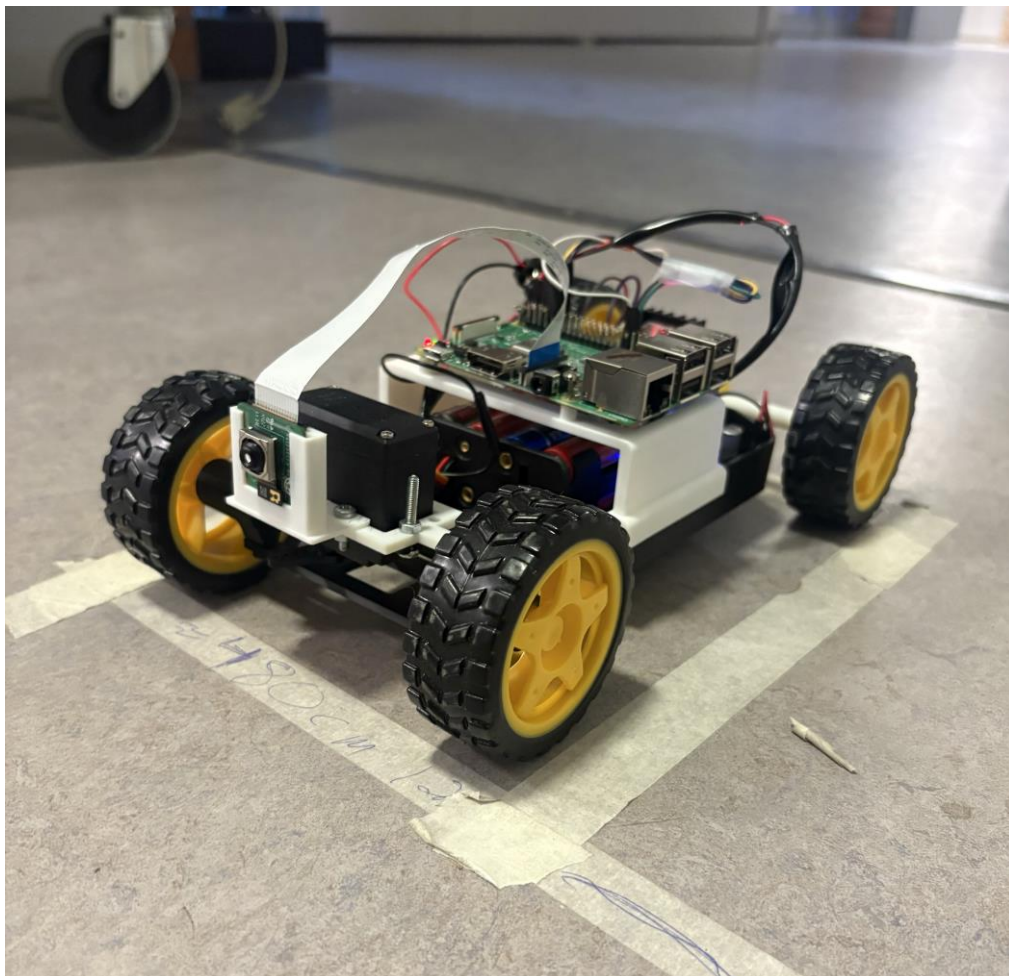
Degree project in Technology

First cycle 15 credits

ConeBot

A camera-controlled robot for autonomous cone navigation

TEESHK NADER



ConeBot

A camera-controlled robot for autonomous cone navigation

Teeshk Nader

Degree project in Mechatronics

Date: 2026-06-05

Supervisor: Filip Strand

Examiner: Daniel Frede

Swedish title: ConeBot

Swedish subtitle: En kamerastyrd robot för autonom konnavigering

Abstract

Camera-based control of an autonomous vehicle is attractive because cameras are inexpensive and provide rich visual information, but it places high demands on fast image processing and stable feedback control. This degree project investigates how limiting the visual update rate affects the stability of a robot car navigating between color-coded cones. A physical test platform was built using a Raspberry Pi 3, a Camera Module 3 and OpenCV. Cones were detected through HSV color filtering, and steering was handled by a PI controller using Cross-Track Error, defined as the horizontal deviation between the image center and the calculated target point between a cone pair. The platform was tested on a 4.8 m track at 5, 10, 15, and 30 FPS. The results show that the maximum stable speed increased from 0.286 m/s at 5 FPS to 0.800 m/s at 30 FPS. They also show that 30 FPS produced a lower CTE spread than 15 FPS despite the higher speed. The results indicate that, within the tested system, the visual update rate and the resulting perception delay were central limitations for stable camera-based navigation.

Keywords

Autonomous navigation, perceptual latency, Raspberry Pi, PI control, Mechatronic, OpenCV.

Sammanfattning

Att styra ett autonomt fordon med kamera är attraktivt eftersom kameror är kostnadseffektiva och ger rik visuell information, men det ställer höga krav på snabb bildbehandling och stabil återkopplad reglering. I detta examensarbete undersöks hur en begränsad visuell uppdateringsfrekvens påverkar stabiliteten hos en robotbil som navigerar mellan färgkodade koner. En fysisk testplattform byggdes med Raspberry Pi 3, Camera Module 3 och OpenCV. Konerna identifierades genom färgfiltrering i HSV-färgrymden, och styrningen utfördes med en PI-regulator baserad på Cross-Track Error, definierat som den horisontella avvikelsen mellan bildens mittlinje och den beräknade målpunkten mellan ett konpar. Plattformen testades på en 4,8 meter lång bana vid 5, 10, 15 och 30 FPS. Resultaten visar att den maximala stabila körhastigheten ökade från 0,286 m/s vid 5 FPS till 0,800 m/s vid 30 FPS. De visar också att 30 FPS gav lägre CTE-spridning än 15 FPS trots högre hastighet. Resultaten indikerar att den visuella uppdateringsfrekvensen och den resulterande perceptuella fördröjningen var centrala begränsningar för stabil kamerabaserad navigering i det testade systemet.

Nyckelord

Autonom navigering, perceptuell latens, Raspberry Pi, PI-reglering, Mekatronik, OpenCV.

Acknowledgments

I would like to express my sincere gratitude to my supervisor Filip Strand for his valuable guidance, support, and feedback throughout the project. I would also like to thank the school's 3D-printing resources, which made it possible to manufacture and improve the mechanical parts of the project, as well as the teaching assistants for practical help and technical support. I am also grateful to other students for ideas, discussions, and encouragement along the way. Finally, AI-based tools were used as limited support for code debugging, code commenting, language polishing, and structuring parts of the written report. All final design decisions, experimental work, tests, analyses, and conclusions in this work are my own.

Stockholm, May 2026
Teeshk Nader

Table of Contents

1	Introduction	1
1.1	Background	1
1.2	Problem	1
1.3	Purpose	1
1.4	Goals	1
1.5	Method.....	2
1.6	Delimitations.....	2
2	Theoretical Background.....	4
2.1	Components	4
2.1.1	Microcontroller and Camera Sensor	4
2.1.2	Steering and Servo Motor (PWM)	4
2.2	Hardware Limitations.....	4
2.3	The Problem of Perceptual Latency.....	4
2.4	The Importance of Look-Ahead Distance	4
2.5	Computer Vision and Color Filtering	5
2.5.1	Software Libraries	6
2.6	Control Engineering and the PID Controller	6
2.7	Summary	7
3	Method	8
3.1	Research Process	8
3.2	Hardware Design and Components	8
3.2.1	Mechanical Design and CAD.....	8
3.2.2	Electrical Components.....	8
3.2.3	Power Supply and Voltage Regulation	9
3.3	Software Design	11
3.3.1	Overview and Program Flow	11
3.3.2	Image Processing and Cone Detection	12
3.3.3	Cone Pair Matching and Look-Ahead Calculation.....	12
3.3.4	PI Controller and Steering Output	14
3.3.5	Limiting the Update Rate and Data Logging.....	14
3.4	Test Environment.....	15
3.5	Data Collection Method (Measurement of FPS and Maximum Speed).....	15
3.6	Validity and Reliability	16
3.6.1	Reliability.....	16
3.6.2	Validity	16
4	Results and Analysis	18
4.1	Maximum Stable Speed at Varying FPS	18
4.2	Control Stability via Cross-Track Error	20
4.3	Discussion	22
5	Conclusions and Future Work	24
5.1	Conclusions.....	24
5.2	Limitations	24
5.3	Future Work	25
	References	26
	Appendix A: Source Code	27
	Appendix B: MATLAB	38
	Appendix C: Video Clips.....	43

List of Figures

Figure 2-1: Illustration of the HSV color space, where hue is separated from brightness.....	5
Figure 3-1: Chassis	8
Figure 3-2: Raspberry Pi 3B and Camera Module 3.....	9
Figure 3-3: MG996R servo motor for steering and the DC motor for rear-wheel drive	9
Figure 3-4: L298 H-bridge and voltage converter	9
Figure 3-5: Wiring diagram for the electrical components	10
Figure 3-6: Flowchart	11
Figure 3-7: Color filtering, HSV	12
Figure 3-8: Diagonal cone-pair mismatch before adjustment (left) and corrected cone- pair selection after adjustment (right).....	13
Figure 3-9: Test Track.....	15
Figure 3-10: Overlaps	15
Figure 4-1: Relationship between frame rate and maximum stable driving speed.....	19
Figure 4-2: Maximum stable PWM value for the drive motor at each frame rate	19
Figure 4-3: CTE over time at 5, 10, 15, and 30 FPS	20
Figure 4-4: Standard deviation of CTE at each frame rate	21
Figure 4-5: Distribution of CTE values at each frame rate.....	22
Figure 5-1: Rear axle after the test period	24

List of Tables

Table 2.1: Summary of theoretical concepts and applied methods	7
Table 4-1: Maximum driving speed and CTE metrics at the four tested frame rates	17

List of Acronyms and Abbreviations

Acronym	Meaning
CAD	Computer-Aided Design
CTE	Cross-Track Error
DC	Direct current
FPS	Frames per second (FPS)
HSV	Hue, Saturation, Value (HSV) - a color space
OpenCV	Open Source Computer Vision Library
PI / PID	Proportional-Integral-(Derivative) controller
PWM	Pulse Width Modulation (control signal to motors)
ROI	Region of Interest in an image

1 Introduction

This chapter defines the subject area and the specific research problem concerning system latency in resource constrained embedded systems. It then presents the purpose of the work, the research questions, the applied methodology, and the delimitations of the study.

1.1 Background

Developing self-driving vehicles is a complex engineering challenge. Many commercial systems rely on advanced and costly sensor technologies such as LiDAR. However, there is considerable scientific and economic value in being able to navigate autonomous systems using only camera-based computer vision [1]. Such a shift from hardware-dependent to software-driven perception, however, places enormous demands on the underlying image processing algorithms and software.

This bachelor's thesis is a direct continuation of an earlier KTH project called "Robocar" [2]. In that project, a robot car equipped with a Raspberry Pi and a camera was built to follow a color-coded object on a treadmill. The system showed good performance at low speeds. However, a fundamental problem was identified when the speed exceeded 8 km/h. At these speeds, strong oscillations occurred in the steering, causing the system to become unstable. This project takes the problem one step further. Instead of object following, a system for dynamic navigation through a track marked by cones is constructed.

1.2 Problem

The core problem is the perceptual latency in the computer vision system. The system is controlled by a Raspberry Pi 3 Model B V1.2, where the camera sensor delivers images at a limited update rate (FPS). In addition, the processor requires time to execute the OpenCV algorithms, including identifying color codes, defining bounding rectangles around the cones, and finally calculating the required steering angle [2].

During this time, the vehicle continues moving forward without access to new steering signals. When the computation is completed and a steering signal is finally sent to the steering servo, the correction is based on outdated visual data. The vehicle is therefore already in a new physical position. This causes the controller to overcompensate, creating severe oscillations that ultimately lead to system collapse at higher speeds [3].

The problem can therefore be summarized in the following question: How does a reduced visual update rate, and the resulting perception delay, affect the vehicle's ability to steer stably?

1.3 Purpose

The purpose is to develop a robot platform and analyze how camera-based navigation is affected by latency, with the aim of determining the highest stable driving speed and evaluating the potential for cost effective autonomous safety technology.

1. How do the tested combinations of frame rate and maximum stable vehicle speed affect the system's Cross-Track Error during camera-based cone navigation?
2. How does the camera's update rate (FPS) affect the vehicle's stability, and what relationship exists between the system's FPS and the highest speed that the vehicle can handle without losing control?

1.4 Goals

The overall goal of this degree project is to design and evaluate a low-cost system for camera-based navigation between cones, and to determine the system's speed limitations related to perceptual latency.

This has been divided into the following three sub-goals:

1. Implement a robust image processing algorithm in OpenCV to identify cones, calculate a midpoint and extract the system's lateral deviation, defined as Cross-Track Error (CTE)
2. Construct a mechatronic test platform with a Raspberry Pi, motor control, and a PI controller, where the derivative action is excluded to avoid instability caused by mechanical vibrations.
3. Experimentally evaluate the relationship between the camera's update rate (FPS) and the vehicle's maximum manageable speed. This is done by progressively limiting the system's FPS and measuring the speed limit before the PI controller loses stability on the test track, to determine how the stability limit changes with the visual update rate.

1.5 Method

To answer the research questions, a physical robot car is designed and evaluated. All tests are performed indoors on a flat floor to eliminate external disturbances. The experimental procedure is divided into two phases:

- **Step 1: Basic navigation.** The computer vision system (OpenCV) processes the camera's image stream and identifies cones as rectangles (bounding boxes). A line is drawn virtually between these rectangles to locate the midpoint. A vector is then calculated from the vehicle's current position to this midpoint. The horizontal deviation between the image center and the calculated midpoint is defined as Cross-Track Error (CTE). This deviation is used as the error signal in the control loop.
- **Step 2: Testing and analysis.** To investigate how latency affects steering, the camera's update rate (FPS) is reduced to fixed levels such as 5, 10, 15, and 30 FPS. For each FPS level, the speed was gradually increased until the PI controller can no longer keep the car stable and it leaves the track. In this way, the highest stable speed for each FPS level can be determined and compared to analyze how the visual update rate affects the vehicle's stability limit.

1.6 Delimitations

This study is limited to a single low-cost prototype based on a Raspberry Pi 3 Model B and a Raspberry Pi Camera Module 3. The results should therefore not be interpreted as general performance limits for all camera-based autonomous vehicles.

The vehicle uses only a forward-facing monocular camera for perception. No LiDAR, radar, IMU, wheel encoders, or external motion tracking system were used. The study therefore focuses on camera-based cone detection and does not evaluate sensor fusion or global positioning.

All tests were carried out indoors on a flat floor under relatively stable lighting conditions. Outdoor driving, uneven surfaces, changing weather, and strong natural light variations were outside the scope of the project.

The camera was fixed to the chassis and was not mechanically stabilized. Mounting the camera on a servo to actively point toward upcoming cone pairs was considered as future work but was not implemented.

The experiments were limited to four visual update rates: 5, 10, 15, and 30 FPS. The study measured the maximum stable speed at each of these frame rates but did not test a full matrix of speeds and FPS levels.

Finally, the implemented controller was limited to a PI controller. More advanced control methods, such as full PID tuning, model predictive control, or adaptive control, were not evaluated.

2 Theoretical Background

This chapter addresses the interaction between the system hardware, image processing algorithms, and control theory. The technical limitations are analyzed to explain the system's behavior under delayed steering signals.

2.1 Components

Autonomous navigation systems are based on technical interaction between several different hardware components. Sensors are used to observe the environment, and a computing unit analyzes the collected data. The decisions are then translated into physical movement via various types of actuators and motors [4].

2.1.1 Microcontroller and Camera Sensor

A computer system for autonomous navigation requires sufficient processing power to perform real-time image analysis. A Raspberry Pi is a type of single board computer that provides a complete Linux environment and sufficient computational capacity to handle complex computer vision algorithms [5]. To perceive its surroundings, the system also requires a camera sensor. The sensor captures light and converts it into digital image streams that can then be processed by the single board computer [6].

2.1.2 Steering and Servo Motor (PWM)

To convert digital steering signals into physical steering angles, servo motors are commonly used. A servo motor is controlled using a technique called pulse width modulation (PWM). This means that the microcontroller sends a square wave in which the pulse length determines the angle to which the servo should move [7]. This enables continuous positioning of mechanical parts.

2.2 Hardware Limitations

Embedded systems for autonomous navigation face several physical and computational limitations that affect control performance. A central challenge is the balance between available computing power and the need for real-time processing. Single board computers have limited clock frequency and memory capacity, which sets an upper limit on how quickly complex image processing algorithms can be executed. A common computing unit for such purposes, the Raspberry Pi 3 Model B, for example has a specified clock frequency of 1.2 GHz. Large amounts of data from camera sensors require extensive resources, which leads to a delay between the moment an image is captured and the moment a steering signal can be generated.

In addition to the computational limitations, mechanical factors in the system actuators must also be considered. Electric motors exhibit physical inertia, which means that it takes time for a vehicle to change its state of motion. Similarly, servo motors introduce a mechanical delay, because the motor and the internal gears need time to physically turn the wheel suspension into an angle. Together, the computation time and the mechanical motion create a total system latency, which places demands on the control system to handle outdated information to maintain stability.

2.3 The Problem of Perceptual Latency

Khalil and Kwon describe how perceptual latency is influenced by the complexity of the incoming image stream, that is, the continuous sequence of camera images that the system must analyze [8]. This means that the latency is not constant but varies depending on the environment and the amount of processing power required to extract relevant features from the image stream [8].

In dynamic navigation situations, rapid feedback is essential. A delay in image processing means that an autonomous vehicle in practice navigates based on outdated information, since the vehicle has time to move a physical distance while the computation is taking place [8]. This shift between the vehicle's perceived position and its actual position reduces the timing margin of the feedback loop. In practice, this means that the controller has less time to correct an error before the vehicle has already moved to a new position. If the correction arrives too late, it can overshoot and increase the steering error instead of reducing it. The controller may therefore react to a part of the track that the vehicle has already passed, which increases the risk that the system will drift off course or become unstable at higher speeds [8].

2.4 The Importance of Look-Ahead Distance

An established way to reduce the effect of image processing latency in camera-based steering is to use a look-ahead distance, that is, to steer toward a point farther ahead in the direction of the track. Taylor et al. show that the need for a longer look-ahead distance increases with speed, because otherwise the vehicle reacts too late to changes in road geometry [3].

If the steering algorithm instead uses a point very close to the vehicle, the system becomes sensitive to small measurement variations, which amplifies the consequences of even short delays. By moving the analysis point forward, a more damped and predictable steering response is created, because the vehicle then reacts earlier to the road curvature and does not need to make equally strong corrections.

This means that turns are initiated in good time, which in practice gives the image processing on the Raspberry Pi unit a larger time window in which to operate. The choice of look-ahead distance therefore becomes a central parameter for achieving stable driving at higher speeds.

2.5 Computer Vision and Color Filtering

Computer vision is a field of technology concerned with teaching computer systems to perceive and interpret information from digital images [9]. One of the most common problems in image processing is that lighting conditions change continuously. Shadows and glare make the same physical object appear completely different to a camera sensor. In the standardized RGB color format (Red, Green, Blue), color information is tightly coupled to brightness. If an object falls into shadow, the red, green, and blue values all decrease simultaneously. This makes it very difficult for an algorithm to determine whether the object still has the same color [10].

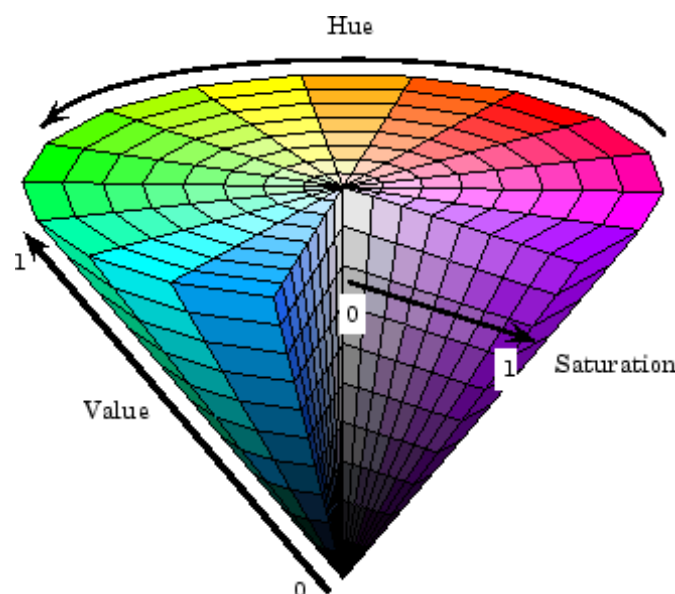


Figure 2-1: Illustration of the HSV color space, where hue is separated from brightness [10].

To solve this problem, the image is often converted from RGB to another color space called HSV (Hue, Saturation, Value), illustrated in Figure 2-1. This model separates the underlying color from the amount of light in the environment.

Hue describes the object's color independently of the lighting. Saturation describes how intense the color is, and Value describes how light or dark it is [10]. By searching only for a specific hue, a system can identify an object with high reliability regardless of whether it is in strong sunlight or deep shadow. This makes HSV color filtering a very robust method for visual recognition in changing environments [10].

2.5.1 Software Libraries

To build the autonomous system, software is needed that can handle image acquisition, image processing, and motor control. This is most easily done with software libraries.

OpenCV (Open Source Computer Vision Library) is a library for computer vision. It contains readymade functions for filtering colors, identifying objects, and calculating reference points in camera images [11].

GStreamer is used to manage the video stream from the camera sensor. It is a media framework based on sending data through a chain of processing steps. Through this structure, video data can be captured from the camera, converted to a new color format, and then sent directly on to image processing [12].

To translate the software's decisions into physical movement, reliable control of the minicomputer's inputs and outputs is required. The pigpio library makes it possible to control the minicomputer's GPIO pins (General Purpose Input/Output) directly. This type of control is needed to generate the pulse width modulation (PWM) that in turn regulates the vehicle's motors [13].

2.6 Control Engineering and the PID Controller

Control engineering is about using feedback to make a system reach and maintain a desired state. In a feedback controller, the difference between the desired target value and the current measured value is called the error. The controller uses this error to calculate how the system should respond to reduce the deviation. A common feedback controller is the PID controller, which combines three terms: proportional, integral, and derivative control. These terms can be interpreted as acting on the present error, the accumulated past error, and the predicted future change of the error, respectively [14].

The proportional term gives a response that is directly related to the current size of the error. The integral term accumulates previous errors over time and is used to reduce small persistent offsets that the proportional term alone may not remove. The derivative term reacts to how quickly the error changes and can therefore help with damp oscillations. However, the derivative term is also sensitive to measurement noise, since rapid noise fluctuations can be interpreted as large changes in the error signal. The ideal continuous-time PID controller can be described as [14]:

$$u(t) = K_p e(t) + K_I \int_0^t e(\tau) d\tau + K_D \frac{de(t)}{dt} \quad (1)$$

2.7 Summary

This chapter has described the theories and technical limitations that affect how an autonomous vehicle can steer itself. The greatest challenge in achieving stable driving is the time delay (latency). This delay is due partly to the computer needing time to analyze the images and partly to the motors having a certain mechanical response time. A summary of these theoretical methods is presented in Table 2.1.

Table 2.1: Summary of theoretical concepts and applied methods.

Method / Concept	Source	Function and benefit in the project	Limitation / Drawback
Computer vision and color filtering (HSV)	[10]	Detects objects reliably even when the room lighting changes.	Requires significant computing power, which delays the image update.
Image processing delay	[8]	Explains why the vehicle has time to travel a distance before it turns.	Causes the control system to act continuously on an outdated view of the track.
Look-ahead	[3]	Compensates for the delay by allowing the car to plan and start the turn earlier.	If the vehicle aims too far ahead, it may cut corners and miss very sharp turns.
Feedback control and PID theory	[14]	Explains how an error signal can be converted into a control signal using proportional, integral, and derivative terms. This provides the theoretical basis for using CTE as input to the steering controller.	Requires careful tuning, and the derivative term can amplify noise in a rapidly changing measurement signal.

3 Method

This chapter describes how the construction and testing were carried out in practice to answer the research questions.

3.1 Research Process

The project is based on an experimental method. The work began by building a physical prototype of the robot car. The code was then written in C++ with OpenCV so that the camera could find the cones and steer the wheels. Finally, practical tests were carried out to collect measurement data on how the image update rate affects the car's speed.

3.2 Hardware Design and Components

The robot car is built from both readymade electronic components and custom designed 3D-printed plastic parts. This section explains how the car is constructed mechanically and electrically.

3.2.1 Mechanical Design and CAD

The car's bottom plate and steering are initially based on a readymade 3D model from the Thingiverse platform [15]. Because this project uses larger electronic components than the original model, the parts were adapted in the Solid Edge software. The bottom plate was enlarged to accommodate a Raspberry Pi. The modified chassis design is shown in Figure 3-1. The vehicle's total weight with all components and batteries mounted was measured at 645 grams. The bracket for the steering servo was redesigned entirely from scratch to fit the chosen servo motor. The new servo mount was also designed to integrate the camera mount, ensuring that both the motor and the camera sensor are stably mounted in a single unit. All modified parts were then manufactured using a 3D printer.

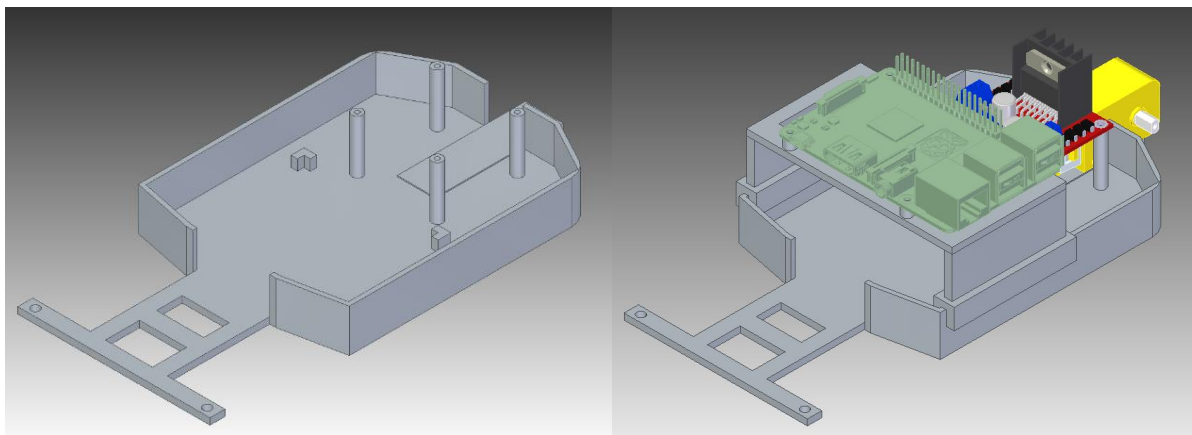


Figure 3-1: Chassis [own screenshot].

3.2.2 Electrical Components

The main computing and perception units are shown in Figure 3-2. The main unit is a Raspberry Pi 3 Model B V1.2, which continuously observes the track via a Camera Module 3. The drivetrain and steering actuators are shown in Figure 3-3. The car is rear-wheel driven, and propulsion is provided by a TT-type DC motor. The front wheels are steered by an MG996R servo motor via a conventional steering mechanism. The motor driver and voltage converter are shown in Figure 3-4. An L298 H-bridge is used as the motor driver so that the microcomputer can regulate the drive motor.

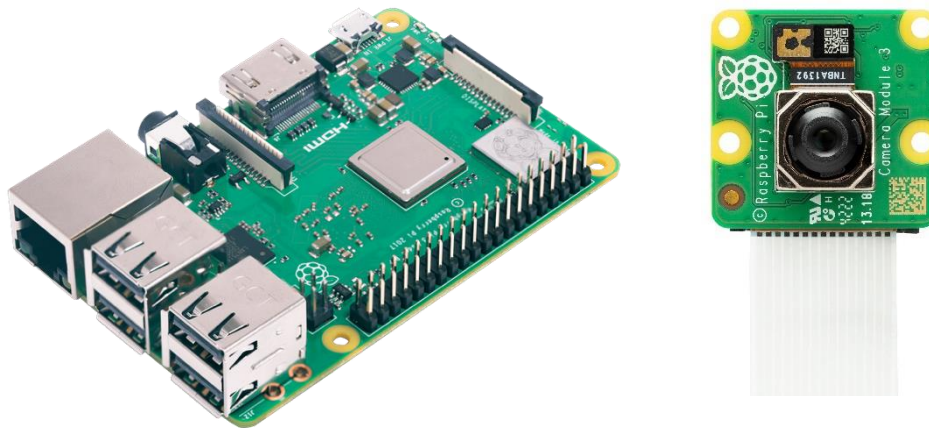


Figure 3-2: Raspberry Pi 3B and Camera Module 3 [5], [6].

To convert the computer's decisions into physical motion, two different motors are used. The car is rear-wheel driven and receives its propulsion from a standard DC geared motor. The front wheels are in turn steered by the MG996R servo motor.

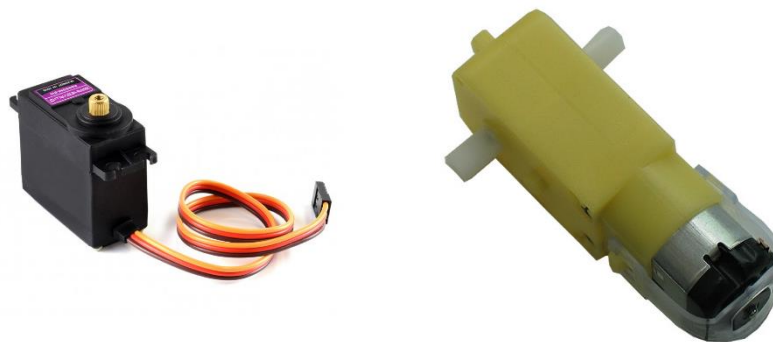


Figure 3-3: MG996R servo motor for steering and the DC motor for rear-wheel drive [16].

Because the minicomputer cannot drive the DC motor directly, a motor driver is required. An L298 H-bridge is used to receive control signals from the computer and convert them into the correct driving force for the rear wheels. In addition, a voltage converter is used to adapt the battery voltage to a safe level for the processor.



Figure 3-4: L298 H-bridge and voltage converter [16].

3.2.3 Power Supply and Voltage Regulation

The power supply consists of eight series-connected alkaline AA batteries, providing approximately 11–12 V. A standard 9 V battery was considered, and had also been used in the earlier Robocar project

[2], but it was not suitable for this prototype because it could not provide enough current under motor load. When the motors started drawing current, the battery voltage dropped, which reduced the available power to the drive motor and made the system less stable.

The Raspberry Pi and the drive motor were supplied through separate branches connected to the same battery pack. The Raspberry Pi received a regulated 5 V supply through a voltage converter, while the motor was driven through the L298 H-bridge. The 5 V supply to the Raspberry Pi therefore did not subtract voltage from the motor branch. However, because all loads were powered by the same battery pack, high motor current could still cause voltage sag in the shared supply. In addition, the L298 H-bridge introduces a voltage drop in the motor path, which further reduces the voltage available to the drive motor. The eight-cell AA battery pack was therefore chosen to provide a higher supply voltage and better current capacity during the short test runs.

The wiring is divided into two circuits to optimize operation. One circuit leads current directly to the H-bridge, which feeds the DC motor with full voltage. The H-bridge's built-in 5-volt output is used specifically to power the servo motor. The second circuit runs from the battery pack to a voltage converter that reduces the voltage to exactly 5 volts for the minicomputer. This separation reduced the risk that motor load variations would disturb the computing unit during testing. A summary of all connections and components is provided in Figure 3-5, where the complete wiring diagram is presented.

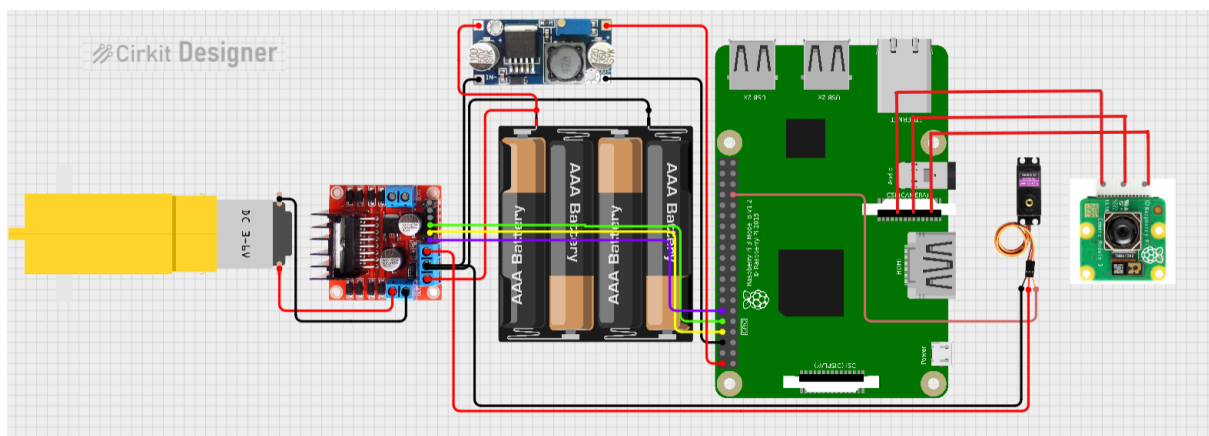


Figure 3-5: Wiring diagram for the electrical components [own figure].

3.3 Software Design

The vehicle's autonomous navigation is controlled by software that analyzes camera images in real-time and converts this information into physical steering signals. This section describes the software architecture from image capture to control output. C++ was chosen because the system's main loop needed to run with low overhead and predictable latency on a resource constrained Raspberry Pi. This made C++ suitable for the time critical combination of image processing, control, and direct hardware communication.

3.3.1 Overview and Program Flow

The overall software loop is shown in Figure 3-6. The program repeatedly captures a frame, filters cones by color, selects a valid cone pair, calculates CTE, and sends a steering command to the servo.

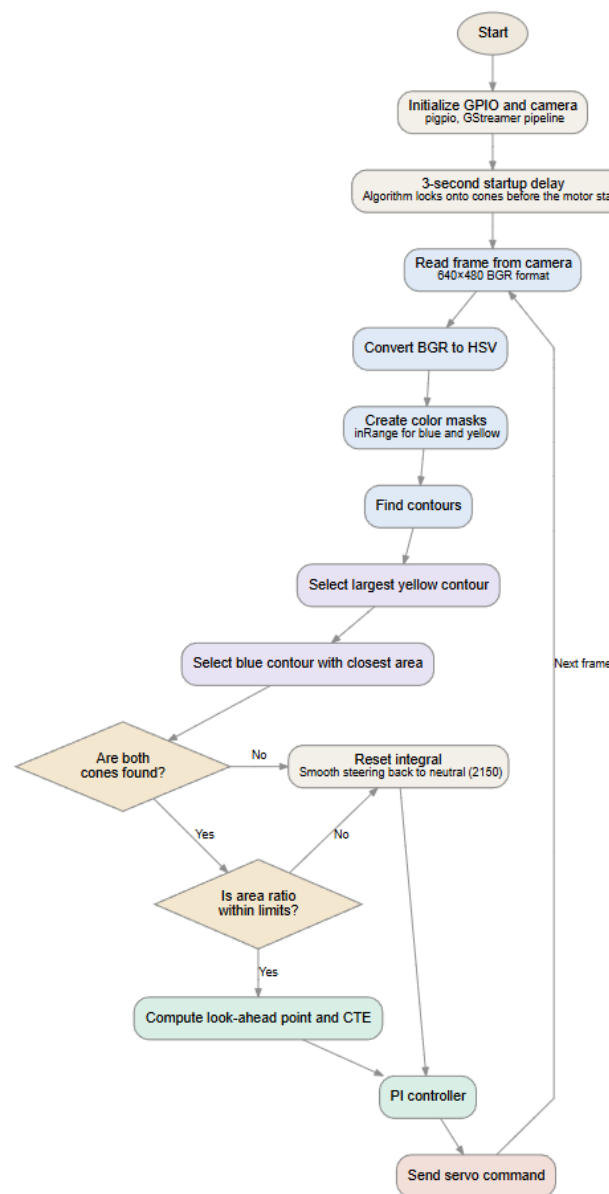


Figure 3-6: Flowchart [own figure].

The software was developed in the C++ programming language using two primary software libraries. OpenCV was used for all image processing and computer vision, while pigpio was used to communicate with the hardware and generate control signals to the vehicle's motors. The camera sensor captures the incoming video stream via a GStreamer pipeline at a resolution of 640 by 480 pixels and a maximum frequency of 30 frames per second.

When the program starts, a delay of three seconds is initiated. This delay gives the algorithm time to process the first images and lock onto a cone pair before the drive motor is activated. For each individual frame, a main loop is then executed that follows a strict sequential process. The image is first color filtered to isolate the cones. A logical check is then performed to pair the correct cones based on their area in the image.

If a valid cone pair is identified, the system calculates a reference point ahead of the vehicle and the corresponding deviation from the image center. This deviation is then fed into a controller that computes the necessary steering correction. If the algorithm fails to identify a valid cone pair, the controller is reset and the steering returns smoothly to a straight-ahead neutral position. Finally, the steering signal is sent to the servo motor and the process is immediately repeated for the next frame. To illustrate this continuous decision-making process, the software's main loop is shown in Figure 3-6.

3.3.2 Image Processing and Cone Detection

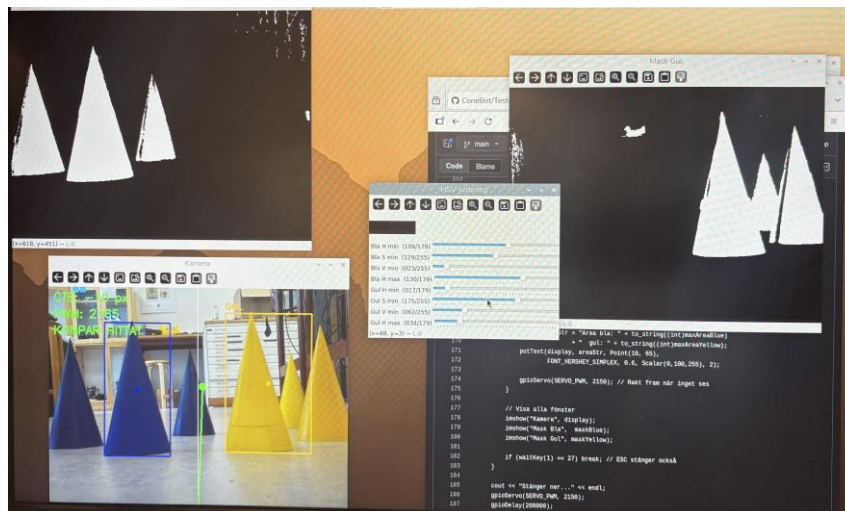


Figure 3-7: Color filtering, HSV [own photo].

When a new frame has been read from the camera, the image processing procedure begins. The incoming video stream is initially delivered in BGR color format. This data is immediately converted to HSV color space. The reason for this choice is that HSV separates the actual color tone from the image's brightness and saturation. This makes the algorithm significantly less sensitive to shadows, reflections, and uneven lighting in the test environment. The cones can therefore be identified with high reliability based solely on their specific color tone.

For the system to distinguish the cones from the background, specific color intervals were calibrated empirically in the test environment. Filtering for the blue cones was set to values between 109 and 130 for hue, 129 to 255 for saturation, and 23 to 255 for value. The yellow cones were calibrated to corresponding values between 17 and 34 for hue, 175 to 255 for saturation, and 142 to 255 for value.

Based on these color ranges, the software creates binary image masks for each color, see Figure 3-7. The algorithm then searches through these masks using the *findContours* function to locate connected clusters of pixels. For each detected pixel cluster, a bounding box is then calculated to define the object's size and position in the image.

To prevent the system from reacting to small patches of color in the background or digital noise in the image, a threshold value is finally applied. A variable called *minArea* was defined as 50 pixels. Any detected rectangles with an area below this threshold are immediately discarded. This filtering ensures that only actual cones are passed on to the next step of the computation.

3.3.3 Cone Pair Matching and Look-Ahead Calculation

Once the cones have been filtered out in the previous step, the software must determine which two cones make up the next physical gate on the track. The algorithm begins this process by identifying the largest yellow cone in the camera's field of view. This specific cone is then used as a reference for the subsequent matching.

Instead of then automatically choosing the largest blue cone, the system calculates the difference in area between the reference cone and all identified blue cones. The blue cone whose area is closest to the yellow cone's area is selected to form a pair. This method minimizes the risk that the algorithm will incorrectly pair a cone near the vehicle with a cone placed much farther away.

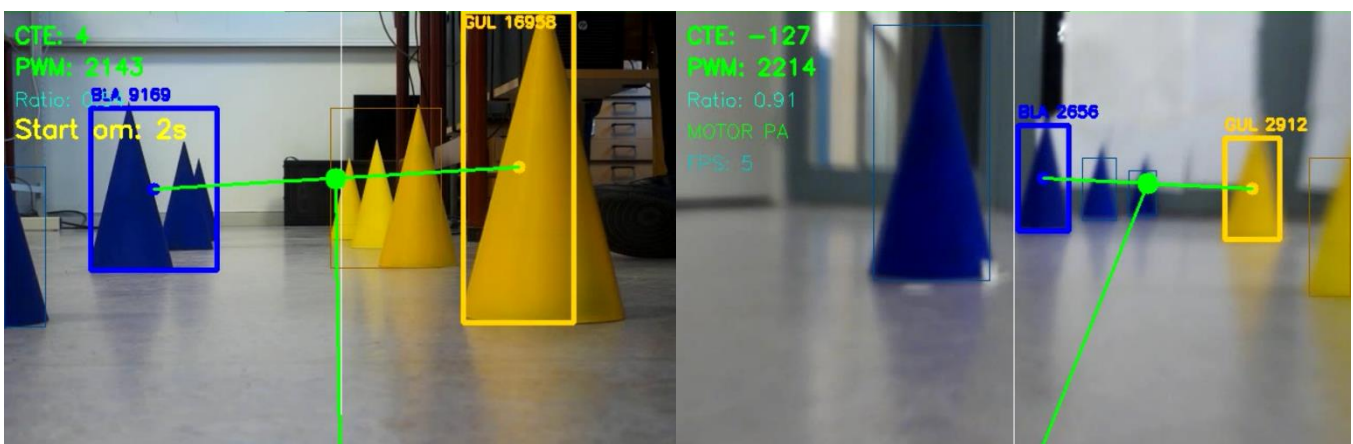


Figure 3-8: Diagonal cone-pair mismatch before adjustment (left) and corrected cone-pair selection after adjustment (right) [own screenshots].

During early tests, the algorithm could sometimes select a blue cone and a yellow cone from different physical gates. In the camera image, this created a false diagonal gate that did not correspond to the actual track opening, as shown in Figure 3-8. This shifted the calculated midpoint away from the real center of the path and produced an incorrect CTE value. To reduce this problem, the final matching method compared cone areas and rejected pairs with an area ratio outside the interval described below. In combination with the final track layout described in Section 3.4, this made the cone-pair selection more consistent.

To further ensure that the selected pair constitutes the next gate, a strict area check was implemented. The system calculates a ratio by dividing the selected blue cone's area by the yellow cone's area. If this ratio exceeds the threshold value 2.5 or falls below 0.4, the pair is rejected entirely. This step prevents the vehicle from reacting to asymmetric pairs in which one cone appears more than 2.5 times larger than the other in the camera image.

When an approved cone pair has been established, the software calculates a central target point ahead of the vehicle. This reference point is often called the look-ahead point and is defined as the geometric midpoint between the bounding rectangles of the two selected cones. Finally, the system's error signal is calculated. This signal is called Cross-Track Error, or CTE. Since the camera resolution has a total width of 640 pixels, the value 320 on the x-axis corresponds to the absolute centerline of the image and the vehicle's straight ahead direction. The steering error is therefore calculated as the horizontal distance in pixels from this centerline to the computed target point.

3.3.4 PI Controller and Steering Output

Once the software has calculated the steering error, this value must be converted into physical motion. This is handled by a PI controller that continuously corrects the vehicle's direction. The following mathematical relationship is used to calculate the required correction:

$$output = K_p \cdot CTE + K_i \int_0^t CTE \, dt \quad (2)$$

In this algorithm, the proportional constant was set to 0.6 and the integral constant to 0.005. These values were selected through empirical tuning during early test runs. The proportional gain was first increased from a low value until the vehicle reacted clearly to lateral error without producing excessive steering oscillations. After that, a small integral gain was added to reduce persistent offset from the track center. The values were not optimized mathematically but were chosen because they gave repeatable and stable behavior on the final test track. The integral is calculated by multiplying the current error by the time difference between two frames and then adding this to a running sum.

Although K_i is numerically much smaller than K_p , the two values are not directly comparable because they act on different quantities. The proportional term is applied directly to the current CTE value in pixels, while the integral term is applied to the accumulated CTE over time. The integral contribution therefore grows if a steering error remains. For this reason, K_i was kept small so that the integral term corrected slow and persistent offsets without dominating the immediate steering response from the proportional term. A larger integral gain would risk delayed overcorrection, especially because the vehicle receives new visual information at a limited frame rate.

The derivative component was deliberately excluded. In theory, the purpose of a derivative term is to damp oscillations by calculating the rate of change of the error signal. However, the chassis construction and the use of standard wheels cause the vehicle to vibrate while driving. In combination with the camera's low frame rate, this causes the cones' bounding boxes to fluctuate in the image stream. According to control theory, differentiating high frequency noise makes the signal tend toward infinity [14]. The result of this phenomenon is often called derivative kick. If a derivative term had been used, the system would interpret these frame-to-frame fluctuations in the detected bounding boxes as large changes in lateral error, which could generate aggressive and unpredictable steering commands. By limiting the controller to a proportional and an integral term, a more stable and predictable physical steering performance was ensured.

Once the controller has calculated the output signal, it is converted into an actual steering command for the servo motor. The vehicle's neutral position for driving straight ahead is defined as the pulse width 2150. The new steering signal is calculated by subtracting the controller's output from this neutral position. To protect the front axle from mechanical damage and prevent the servo from turning too far, the signal is strictly limited to the interval between 1900 and 2400. To further stabilize the vehicle's motion, a smoothing factor of 0.35 is applied. This software filtering combines the previous steering value with the new value. The result eliminates sudden jerks and provides a much smoother transition between different steering angles.

Finally, a safety function was implemented for situations in which the system completely loses its visual reference. If the algorithm fails to find a valid cone pair, the integral sum is immediately reset. The software then forces the steering to return smoothly to the neutral position 2150. This function prevents the car from making unintended sharp turns at the end of the track or if visibility is temporarily blocked.

3.3.5 Limiting the Update Rate and Data Logging

To investigate how the visual update rate affects steering, a function was created that locks the frame rate in the software. This is done by measuring how long each frame takes to process. The system first calculates a target time by dividing 1.0 by the desired FPS value. If the software works faster than this

target time, the program is temporarily paused using the `gpioDelay` command. In this way, the camera can be locked precisely to 5, 10, 15, or 30 frames per second.

The software also contains a system for collecting data while driving. For each frame, important variables are saved, such as the steering error in pixels, the pulse width to the servo motor, and the area ratio underlying cone selection. At the same time, the camera's video stream is recorded locally on the minicomputer using the VideoWriter module. This combined data is then exported manually to MATLAB for further analysis.

3.4 Test Environment



Figure 3-9: Test Track [own photo].

All tests were carried out indoors on a flat floor with even lighting, see Figure 3-9. This was done to eliminate external disturbances from strong sunlight and dynamic shadows that otherwise could have affected the camera sensor negatively.

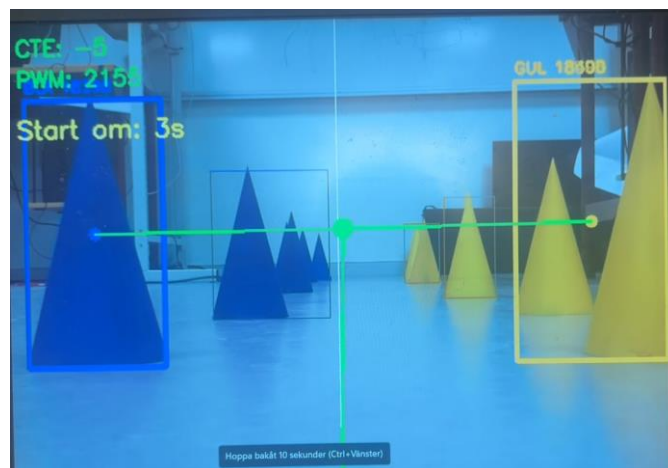


Figure 3-10: Overlaps [own photo].

The initial test plan included an S-shaped track. Practical tests, however, showed that such a design created significant visual overlaps between cone pairs in the camera image. These overlaps, see Figure 3-10, led to incorrect area matching and caused the software to calculate misleading steering errors. To solve this problem, the final test track was instead constructed as a continuous circular arc consisting of a total of five cone pairs. A circular arc minimizes visual overlap between cone pairs and

gives the algorithm a much clearer line of sight, while still providing a sufficient challenge for the vehicle's steering system. The total length of the track was measured at 4.8 meters.

The final track length of 4.8 m was chosen as a practical compromise. A longer track would have allowed more cone transitions to be observed, but the alkaline AA battery pack showed noticeable voltage drop during repeated testing. A longer run would therefore risk the vehicle starting at a higher speed and gradually slowing down before reaching the end of the track. The 4.8 m track was considered long enough to include several steering corrections, while still short enough to keep the vehicle speed approximately constant within each measurement.

The track was marked using large cones with a base diameter of 10 centimeters and a height of 20 centimeters. The choice of large cones was a deliberate design decision. Smaller cones are often misinterpreted by the algorithm as background noise or cause the vehicle to change direction toward the next gate too early. The larger cones occupy a larger portion of the camera's field of view. This forces the vehicle to continue navigating toward the midpoint of the gate for longer before aligning with the next cone pair, which considerably reduces the risk that the car will collide with the cones before it has passed through the gate.

The placement of the cones was fine-tuned empirically to optimize the vehicle's navigation conditions. The distance between the blue and yellow cones in each pair was set to approximately 20-30 centimeters to give the vehicle enough space to pass freely. The distance between the different cone pairs along the track was set to about 70-90 centimeters. This larger space was chosen specifically to give the vehicle enough time to adjust its direction. If the cone pairs had been placed too close together, the vehicle would have been forced into constant and jerky direction changes, which would have increased the risk that the system would become unstable and crash.

3.5 Data Collection Method (Measurement of FPS and Maximum Speed)

To evaluate how the visual update rate affects the vehicle's speed, a stress test was carried out. The frame rate was systematically locked to 30, 15, 10, and 5 frames per second. For each frequency, the vehicle's motor speed increased stepwise. The criterion for a successful test was that the vehicle should be able to navigate the entire track without crashing. The highest setting that the vehicle could complete stably three times in a row was recorded as the maximum limit.

The experiment did not measure the complete end-to-end latency from image capture to physical steering response directly. Instead, the camera update rate was used as a controlled experimental variable. A lower frame rate increases the time between new visual measurements and therefore increases the amount of outdated visual information available to the controller. In this report, perceptual latency is therefore used to describe the practical delay introduced by slower visual updates and image processing in the control loop.

To obtain objective measurements, the actual physical speed was measured. This was done by timing the 4.8-meter track with a stopwatch. The speed was then calculated in meters per second by dividing the distance by the time.

3.6 Validity and Reliability

To ensure that the measurements of the vehicle's performance are scientifically sound, both reliability and validity were considered when designing the tests.

3.6.1 Reliability

To ensure repeatable measurements, each approved speed test was performed three times. The speed level was recorded as stable only when the vehicle completed the same setting in all three runs. For the detailed CTE analysis, one representative run per FPS level was then used. Another critical factor

was the power supply. To prevent declining battery voltage from causing inconsistent motor performance, only alkaline batteries measuring above 1.4 V before testing were used. This ensured sufficiently stable conditions throughout all measurement series.

3.6.2 Validity

The camera's two-dimensional perspective in curves can create measurement errors that risk being misinterpreted as system instability. To avoid this, the test track was designed as a circular arc with large cones, which minimized visual overlap. In addition, the yellow cone was placed slightly farther forward than the blue one in each cone pair. This offset counteracted perspective errors and ensured that the algorithm consistently calculated correct reference points. Finally, the actual physical speed in meters per second was measured instead of relying on the motor's digital control signal. This ensured that the experiment evaluated exactly the relationship between frame rate and performance formulated in the research questions.

It should also be noted that CTE was measured internally from the camera image rather than by an external motion tracking system. The CTE values therefore represent the controller's perceived lateral error, not an independently measured ground-truth position of the vehicle. This is still relevant for evaluating the control loop, since the controller acts directly on this perceived error signal, but it limits how precisely the values can be interpreted as physical lateral displacement.

4 Results and Analysis

This chapter presents and analyzes the results produced by the stress test described in Section 3.5. The results are presented in two parts. First, the maximum stable driving speed achieved by the vehicle at the four different frame rates is discussed. This is followed by an analysis of the system's control stability based on the logged Cross-Track Error (CTE) during driving. Finally, the results are discussed in relation to the research questions stated in Chapter 1.

4.1 Maximum Stable Speed at Varying FPS

Table 4-1 shows that the maximum stable driving speed increased with the camera's frame rate. At 5 FPS, 0.286 m/s was measured, while the corresponding value at 30 FPS was 0.800 m/s. The results therefore show a clear positive relationship between FPS and the highest stable speed.

Table 4-1: Maximum driving speed and CTE metrics at the four tested frame rates.

FPS	PWM	Time (s)	Speed (m/s)	CTE-std (px)	CTE max (px)
5	85	16.80	0.286	58.6	164
10	90	14.16	0.339	72.0	203
15	95	9.00	0.533	92.0	210
30	110	6.00	0.800	73.6	175

These measurements should be interpreted as boundary measurements. For each frame rate, the vehicle was driven at the highest speed that still allowed it to complete the track in a stable manner. The data therefore do not describe the full control behavior at all possible speeds for each FPS setting, but rather the stability limit reached under each tested frame rate. At 5 FPS, the maximum speed was 0.29 m/s. At 30 FPS, the speed limit had increased to 0.80 m/s, corresponding to an increase of approximately 180 percent. This relationship is illustrated in Figure 4-1, where it is evident that the speed limit increases monotonically with FPS, but not linearly. The largest percentage increase occurs between 10 and 15 FPS, where the maximum speed rises from 0.34 m/s to 0.53 m/s.

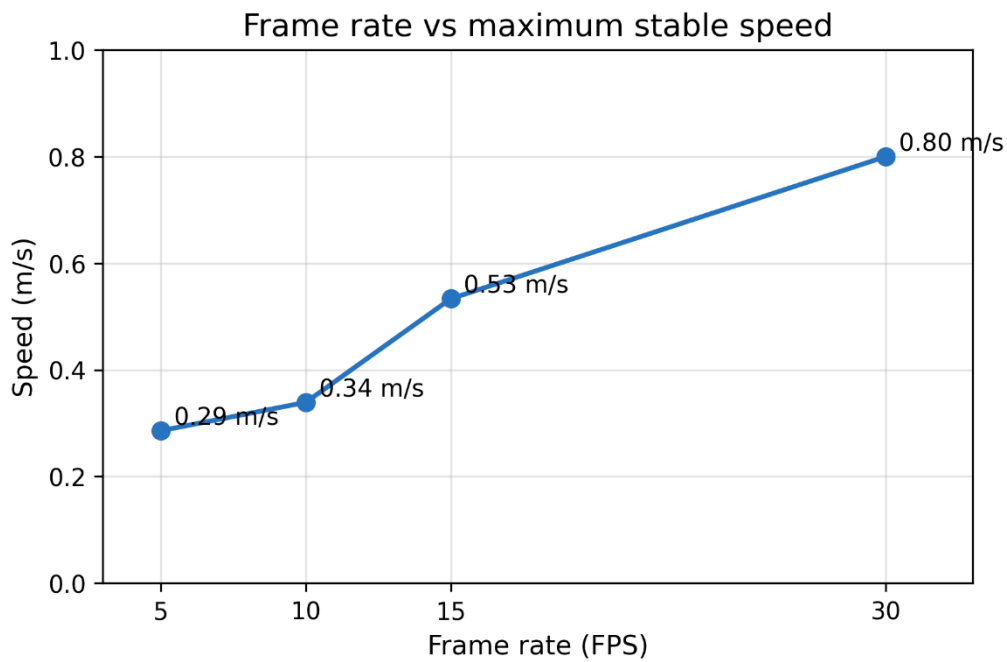


Figure 4-1: Relationship between frame rate and maximum stable driving speed [own figure].

The same trend is reflected in the underlying drive motor control signal shown in Figure 4-2. The PWM value increases from 85 at 5 FPS to 110 at 30 FPS. It is worth noting that the lower limit of 85 PWM corresponds to the drive motor's starting threshold on the current chassis. At lower PWM values, the motor could no longer overcome the vehicle's rolling resistance. This means that the result at 5 FPS is not limited by the controller but by the motor's mechanical starting voltage, which is an important limitation when interpreting the result.

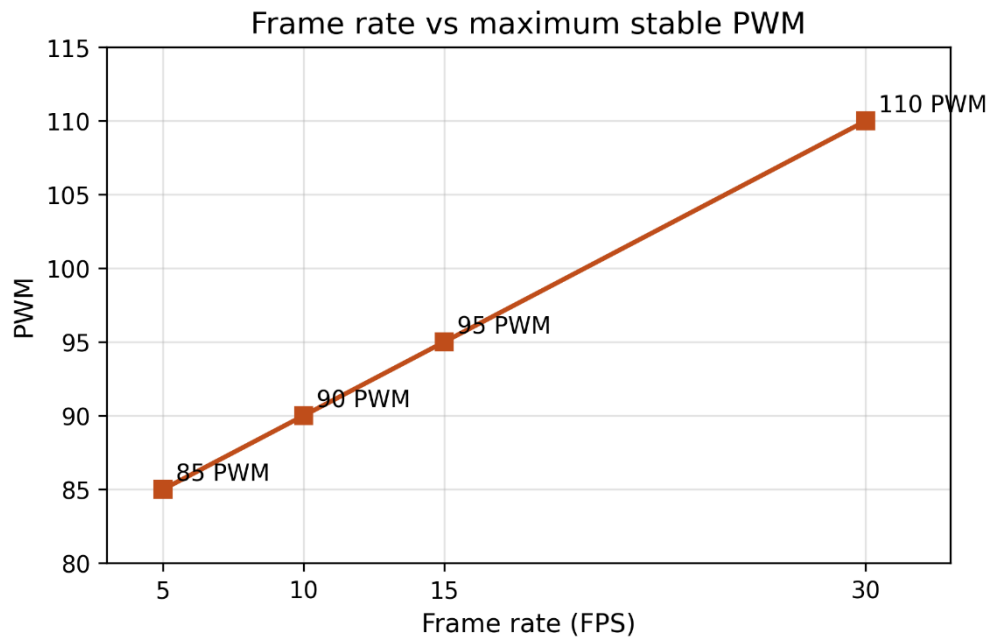


Figure 4-2: Maximum stable PWM value for the drive motor at each frame rate [own figure].

4.2 Control Stability via Cross-Track Error

Figure 4-3 shows the CTE signal at each FPS level, while Table 4-1 and Figure 4-4 summarize the spread of the signal. The results show that control stability does not vary monotonically with speed but is strongly influenced by frame rate. The largest CTE spread was measured at 15 FPS, whereas 30 FPS produced a smaller spread despite a higher driving speed.

The CTE traces in Figure 4-3 were recorded at the maximum stable speed for each frame rate. This means that the figure compares the controller close to its stability limit, rather than comparing all FPS levels at the same vehicle speed. A run with higher FPS at a lower speed would likely produce smoother CTE variation and more frequent smaller corrections, since the system would receive more visual updates per meter traveled. However, this was not measured in the present study and should therefore be treated as expected behavior rather than an experimental result. Since the x-axis represents time, a slower run would be expected to last longer, while a plot normalized by distance would be needed to compare the steering behavior per meter of travel.

Frames in which no valid cone pair was detected were excluded from the CTE statistics. The CTE plots therefore describe the behavior during valid detections, while temporary perception losses are discussed qualitatively as part of the system latency and stability behavior.

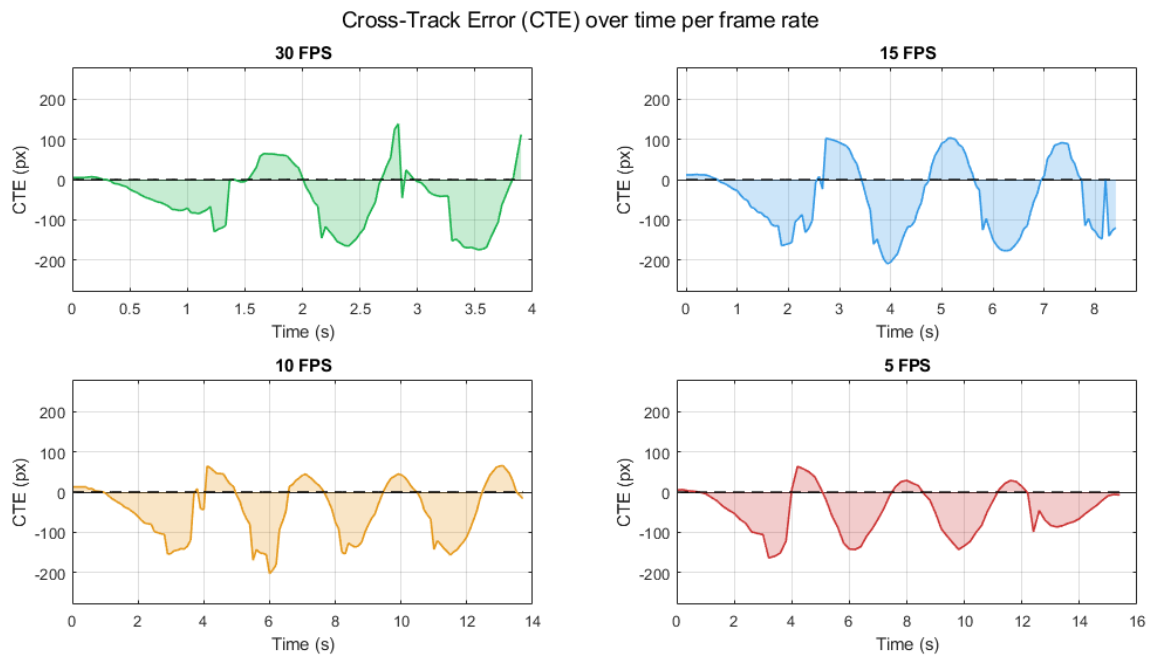


Figure 4-3: CTE over time at 5, 10, 15, and 30 FPS [own figure].

The time series shows that all configurations exhibit oscillations around the zero line, which is expected because the track consists of five cone pairs and the vehicle turns periodically between them. The character of the oscillations, however, differs significantly. At 5 and 10 FPS, the signal shows even and relatively slow oscillations, while at 15 FPS it reaches the largest amplitudes in the entire test, with peak values of +104 and -210 pixels. At 30 FPS, the amplitudes are smaller again, even though the speed is twice as high as at 15 FPS.

All subgraphs in Figure 4-3 also show short sudden jumps in the signal. These correspond to frames in which the algorithm was temporarily unable to pair a cone pair, either because the area ratio fell outside the threshold range 0.4-2.5 or because a cone moved out of the camera's field of view. When this happens, the integral term is reset and the steering returns smoothly toward the neutral position according to the logic in Section 3.3.4. When the algorithm locks onto a valid cone pair again in a subsequent frame, the CTE value suddenly jumps to its new level. Because the vehicle continues

forward during the miss without a new steering correction, these short interruptions also contribute to the total perceptual latency of the system.

To quantify the oscillation level, the standard deviation of CTE was calculated for each FPS level. The result is shown in Figure 4-4. The standard deviation is not monotonic in FPS, which is one of the most central observational results of this study. The lowest spread was measured at 5 FPS (58.6 px), the highest at 15 FPS (92.0 px), and at 30 FPS the spread decreased again to 73.6 px.

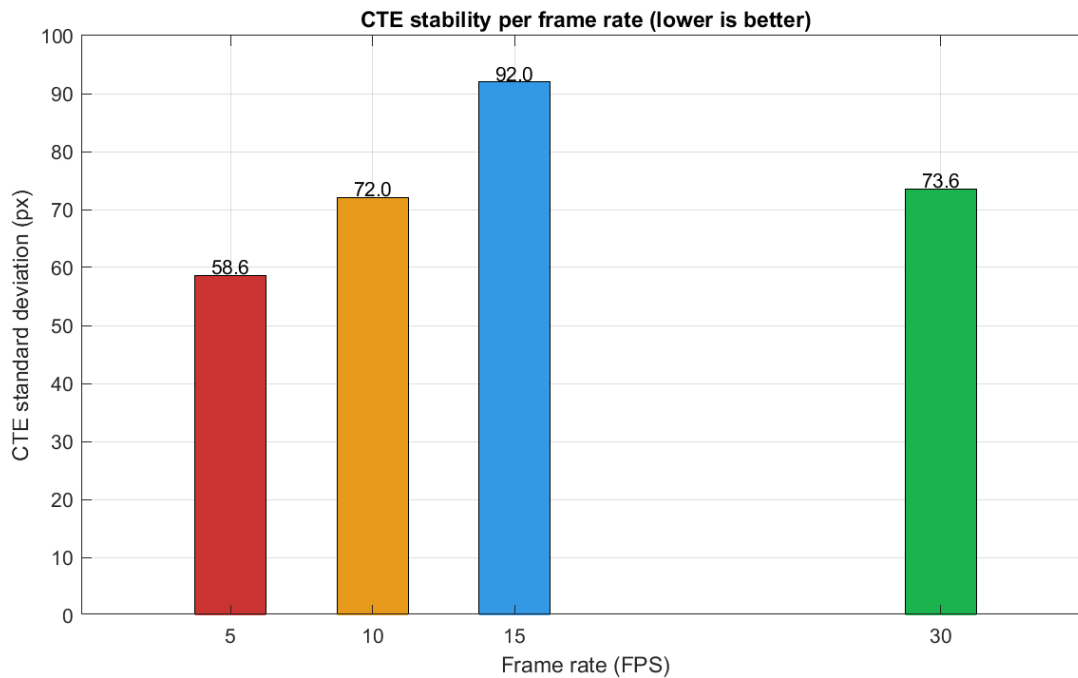


Figure 4-4: Standard deviation of CTE at each frame rate [own figure].

The value at 5 FPS should be interpreted with caution. Because the drive motor was operating at its lower starting threshold and the vehicle was moving at 0.29 m/s, the control loop had time to make several corrections per unit track length despite the low update rate. The low spread therefore does not primarily reflect high control quality, but rather that the vehicle simply moved slowly enough to recover from each delayed steering signal.

The result at 15 FPS is more informative. Here the vehicle travels at 0.53 m/s, which is fast enough for the frame interval of about 67 ms per frame to generate a significant position shift between measurement and steering, but not so fast that the vehicle loses control and leaves the track. The PI controller therefore operates close to its stability limit at this FPS level, which is reflected both in the high standard deviation and in the fact that the amplitudes in Figure 4-3 reach -210 px.

A likely explanation for the peak at 15 FPS is that this test case combined a relatively high vehicle speed with an update rate that was still too low to provide sufficient correction margin. At 5 and 10 FPS the vehicle moved more slowly, which allowed the controller to recover between delayed corrections. At 30 FPS the vehicle moved faster, but the controller received new visual information every 33 ms, which reduced the position error accumulated between frames. At 15 FPS, however, the vehicle was fast enough for the 67 ms update interval to create large position shifts, while the update rate was not high enough to compensate for them. This placed the PI controller close to its stability limit and explains why the largest CTE spread occurred at this frame rate.

The most important single result is that the CTE spread at 30 FPS (73.6 px) is lower than at 15 FPS (92.0 px), despite the speed being 50 percent higher. This indicates that reducing the frame interval from 67 ms to 33 ms gives the control loop enough margin not only to handle the higher

speed, but also to keep the vehicle closer to the center of the lane than during the slower run at 15 FPS. The result contradicts the expectation that higher speed would automatically produce larger deviations, and it supports the interpretation that frame-rate-related perceptual delay was a central factor for stability within the tested configuration.

The distribution of CTE values at each FPS level is further illustrated in Figure 4-5. The histograms show that all configurations have a concentration of values around zero and a second peak in the negative intervals, corresponding to the moments when the vehicle is in a turn. The sign of the CTE should not be interpreted as a direct measure of stability, since positive and negative values mainly indicate which side of the image center the target point is located on. For this reason, the standard deviation and maximum absolute CTE are more suitable measures for comparing control stability between FPS levels. At 30 FPS, the distribution is more concentrated than at 15 FPS, confirming the same conclusion indicated by the standard deviation.

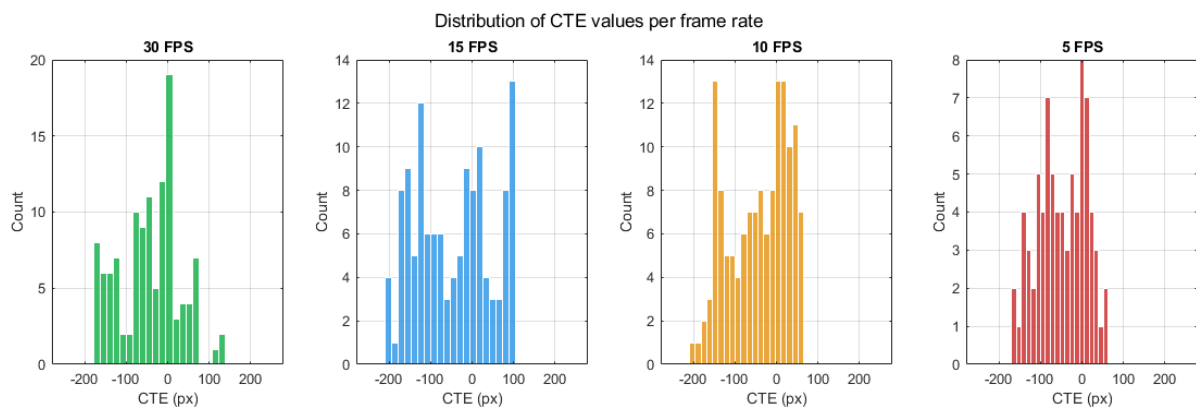


Figure 4-5: Distribution of CTE values at each frame rate [own figure].

The lower CTE spread at 30 FPS suggests that the controller had more correction margin than at 15 FPS, but it does not prove that substantially higher speeds were available without further testing, since the maximum speed was also limited by the motor, battery, steering mechanics, and cone detection reliability.

4.3 Discussion

The results answer the two research questions formulated in Section 1.3. The first question concerned how the tested combinations of frame rate and maximum stable speed affected the system's CTE. Because the measurements were boundary tests, the results should not be interpreted as a complete speed sweep at each FPS level. Instead, they show that CTE behavior cannot be explained by speed alone. The comparison between 15 and 30 FPS is especially important: 30 FPS allowed a higher speed while still producing a smaller CTE spread, which indicates that fresher visual information improved the available correction margin.

The second research question concerned the relationship between FPS and the highest manageable speed. As shown in Figure 4-1, this relationship is monotonically increasing but not linear. The maximum speed increases by a factor of 2.8 when FPS increases by a factor of 6, which suggests that bottlenecks other than camera latency alone begin to dominate at higher FPS. Contributing factors identified during testing include the servo motor's mechanical response time, wheel vibrations in the chassis that cause fluctuations in bounding-box detection, and the vehicle's total weight of 645 grams, which makes steering sluggish in tight turns.

In relation to the initial problem in Section 1.2, where it was noted that the Robocar project collapsed at speeds above 8 km/h, this study provides a possible quantitative perspective. That collapse occurred in a configuration corresponding to the lower end of the FPS range in this study. The results show that a higher visual update rate can shift the stability limit upward. With faster

image updates, the control system has time to correct the vehicle's course earlier, allowing the car to be driven at higher speeds before losing control. This means that the stability of a camera-based control system depends not only on the hardware but can also be substantially improved by increasing the visual update rate.

It is important to emphasize, however, that this study examined only four discrete FPS levels. The exact shape of the relationship curve between these points is therefore not established and based on these data it is also not possible to say what happens above 30 FPS, which is the upper limit of the camera sensor at the chosen resolution. These limitations are left for future work and are discussed further in Chapter 5.

5 Conclusions and Future Work

This chapter summarizes the conclusions of the study. It then describes the limitations that affected the work. Finally, possible directions for future development are proposed.

5.1 Conclusions

Based on the results in Chapter 4, it can be concluded that the vehicle's maximum stable speed increases with the camera's frame rate. The maximum stable driving speed increased from 0.286 m/s at 5 FPS to 0.800 m/s at 30 FPS, while the CTE spread at 30 FPS was lower than at 15 FPS despite the higher speed. However, the relationship between speed and control stability is not one directional. A higher frame rate can compensate for a higher speed, making it possible for the vehicle to drive faster while staying closer to the centerline. The results therefore indicate that, within the tested configuration, frame-rate-related perceptual delay was a central factor limiting stable camera-based control.

5.2 Limitations

Several limitations arose during the project that affected both the implementation and the interpretation of the results. First, the front steering linkage introduced mechanical play. The front wheels were connected using a thin metal wire, which gradually loosened after repeated runs and minor collisions. As a result, the wheels did not always point perfectly forward when the servo returned to its neutral position. This likely contributed to some of the sudden peaks seen in Figure 4-3.

Second, the rear drive axle was undersized relative to the vehicle's total weight of 645 g. During testing, the axle bent slightly, as shown in Figure 5-1. This caused the rear wheels to roll less evenly and may have reduced the maximum stable speed. A stronger drivetrain and a stiffer axle would likely improve the mechanical stability of the platform.



Figure 5-1: Rear axle after the test period [own photo].

Third, there was the power supply. The tests were carried out using eight series connected AA batteries and debugging as well as repeated test runs consumed a great deal of power. The voltage dropped gradually during each test session. When the batteries approached their lower limit, the vehicle's speed decreased automatically until it eventually could no longer move. A rechargeable RC battery would have provided a more stable voltage throughout the entire test session and thereby made the measurements more comparable between runs. This also limited the possible track length. A

longer course would have increased the influence of battery voltage drop within each run, making the comparison between FPS levels less reliable.

Fourth, there were the controller settings. On the one hand, the PI controller values $K_p = 0.6$ and $K_i = 0.005$ worked sufficiently well for the track and the prototype. On the other hand, these values are not optimized, and there is very likely room for improvement through more systematic tuning of the parameters.

Finally, one limitation concerned the measurements themselves. The study tested only four discrete FPS levels, which means that the exact shape of the relationship curve between these points is not established. Furthermore, 30 FPS is the camera sensor's upper limit at the selected resolution of 640×480 , and it is therefore not possible based on this study to say anything about what happens at even higher frame rates. All tests were also performed indoors on a flat floor, which means that the results cannot be directly transferred to outdoor conditions.

5.3 Future Work

The study has opened several natural directions for further development. The most direct step is to address the mechanical limitations in Section 5.2. A thicker spindle wire or an entirely new suspension design would eliminate the play in the front axle, and a more powerful DC motor with a thicker shaft would solve the problem of the bent rear axle. Together, these improvements would probably make higher stable speeds possible at all FPS levels. A useful continuation would be to test a full speed-FPS matrix, for example by running 30 FPS at the same speeds used for 5, 10 and 15 FPS. This would separate the effect of frame rate from the effect of vehicle speed and show whether higher FPS mainly shifts the stability limit upward or also improves CTE behavior at equal speeds.

As regards the power supply and the controller, there is also room for improvement. A rechargeable RC battery would provide a more stable voltage throughout the test session and thereby make the measurements more comparable. A more systematic tuning of the PI controller values K_p and K_i could also improve control performance without any hardware changes.

On the sensor side, one direct possibility is to mount the camera on the micro servo SG90 identified in Section 1.6. A servo driven camera could mechanically stabilize the image in curves and point toward upcoming cone pairs, which would further reduce the effective latency. Replacing the computing unit with a more powerful one, such as a Raspberry Pi 5, would also make it possible to investigate what happens at frame rates above 30 FPS.

Finally, the relationship curve in Figure 4-1 could be mapped in greater detail by testing more FPS levels, especially in the interval between 15 and 30 FPS where the greatest change in control behavior was observed. It would also be valuable to extend the tests to more realistic conditions, for example outdoor environments or tracks with sharper turns.

References

- [1] Y. Wang, W.-L. Chao, D. Garg, B. Hariharan, M. Campbell and K. Q. Weinberger, "Pseudo-LiDAR from Visual Depth Estimation: Bridging the Gap in 3D Object Detection for Autonomous Driving," 2019. [Online]. Available: <https://arxiv.org/pdf/1812.07179> [Accessed: 10 February 2026].
- [2] F. Kulesza and L. Backlund, "Robocar: Objektföljande robot med hjälp av en monokulär kamera," Bachelor's thesis, KTH, Stockholm, Sweden, 2025.
- [3] C. J. Taylor, J. Kosecka, R. Blasi and J. Malik, "A comparative study of vision-based lateral control strategies for autonomous highway driving," *The International Journal of Robotics Research*, vol. 18, no. 5, pp. 442-453, 1999. [Online]. Available: <https://www.cis.upenn.edu/~cjtaylor/PUBLICATIONS/pdfs/TaylorIJRR99.pdf> [Accessed: 10 February 2026].
- [4] E. Yurtsever, J. Lambert, A. Carballo and K. Takeda, "A Survey of Autonomous Driving: Common Practices and Emerging Technologies," *IEEE Access*, vol. 8, pp. 58443-58469, 2020. [Online]. Available: <https://ieeexplore.ieee.org/document/9046805> [Accessed: 10 February 2026].
- [5] Raspberry Pi Ltd., "Raspberry Pi 3 Model B," 2016. [Online]. Available: <https://www.raspberrypi.com/products/raspberry-pi-3-model-b/> [Accessed: 15 February 2026].
- [6] Raspberry Pi Ltd., "Camera Module 3," product specification, 2023. [Online]. Available: <https://datasheets.raspberrypi.com/camera/camera-module-3-product-brief.pdf> [Accessed: 15 February 2026].
- [7] Next.gr, "Controlling Servo with PWM Tutorial," [Online]. Available: <https://www.next.gr/tutorials/motor-control-systems/controlling-servo-with-pwm-tutorial> [Accessed: 15 February 2026].
- [8] A. Khalil and J. Kwon, "PLM-Net: Perception Latency Mitigation Network for Vision-Based Lateral Control of Autonomous Vehicles," arXiv:2407.16740, 2024. [Online]. Available: <https://arxiv.org/abs/2407.16740> [Accessed: 20 February 2026].
- [9] IBM, "What is Computer Vision?," 2024. [Online]. Available: <https://www.ibm.com/topics/computer-vision> [Accessed: 25 February 2026].
- [10] J. Fisher, "Understanding the HSV Color Model," Lifewire, 2023. [Online]. Available: <https://www.lifewire.com/what-is-hsv-in-design-1078068> [Accessed: 29 February 2026].
- [11] OpenCV, "About OpenCV," [Online]. Available: <https://opencv.org/about/> [Accessed: 1 April 2026].
- [12] GStreamer, "What is GStreamer?," [Online]. Available: <https://gstreamer.freedesktop.org/> [Accessed: 10 April 2026].
- [13] J. Garland, "pigpio library," [Online]. Available: <https://abyz.me.uk/rpi/pigpio/> [Accessed: 1 April 2026].
- [14] T. Glad and L. Ljung, *Reglerteknik: Grundläggande teori*, 4th ed. Lund, Sweden: Studentlitteratur, 2006.
- [15] Maker "the_senator", "RC Car Chassis," Thingiverse. [Online]. Available: <https://www.thingiverse.com/thing:6667669> [Accessed: 6 March 2026].
- [16] Electrokit, "Electronic components and product images". [Online], Available: <https://www.electrokit.com>, [Accessed 3 May 2026].

Appendix A: Source Code

```

/*
 * ConeBot main program
 * MF133X KTH
 *
 * Purpose:
 * Detects blue and yellow cones in the camera image, calculates Cross-Track Error (CTE)
 * and controls the vehicle using a PI controller.
 *
 * Main parts:
 * 1. Initialization of GPIO, camera and video recording
 * 2. Color filtering in HSV and cone detection
 * 3. Matching cone pairs and calculating the target point
 * 4. PI control and steering signal to the servo
 * 5. FPS limiting, logging and safe shutdown
 */

// Libraries:
#include <opencv2/opencv.hpp>
#include <pigpio.h>

#include <chrono>          // time measurement between frames
#include <iostream>        // print text to terminal
#include <csignal>         // signal handling such as Ctrl+C
#include <ctime>           // date and time for unique video filename

// Avoid long prefixes:
using namespace std;
using namespace cv;

#define MOTOR_PWM 18
#define MOTOR_HIGH 14
#define MOTOR_LOW 15
#define SERVO_PWM 13

volatile sig_atomic_t keepRunning = 1; // global run flag

```

```

// Handle safe shutdown on Ctrl+C, prevents abrupt exit
void stopProgram(int signum) {
    cout << "\n[Ctrl+C] Exiting..." << endl;

    keepRunning = 0;
}

// Shut down motor, servo and GPIO safely so the car does not remain
// with an active motor signal if the program ends
void safeShutdown() {
    gpioWrite(MOTOR_HIGH, 0);
    gpioWrite(MOTOR_LOW, 0);
    gpioPWM(MOTOR_PWM, 0);
    gpioServo(SERVO_PWM, 2150);
    gpioDelay(300000);
    gpioTerminate();
}

int main() {
    // Configure pigpio so the program handles Ctrl+C itself.
    // Important because we want to safely stop motors and servos
    // before the program exits.
    int cfg = gpioCfgGetInternals();
    cfg |= PI_CFG_NOSIGHANDLER;
    gpioCfgSetInternals(cfg);
    signal(SIGINT, stopProgram);

    if (gpioInitialise() < 0) {
        cerr << "Failed to start pigpio." << endl;
        return -1;
    }

    // Configure GPIO for motor and servo as outputs
    gpioSetMode(MOTOR_HIGH, PI_OUTPUT);
    gpioSetMode(MOTOR_LOW, PI_OUTPUT);
    gpioSetMode(MOTOR_PWM, PI_OUTPUT);
    gpioSetMode(SERVO_PWM, PI_OUTPUT);

    gpioWrite(MOTOR_HIGH, 0);           // Before driving, set all control signals

```

```

gpioWrite(MOTOR_LOW, 0);          // to a safe initial state. Prevents the car
gpioPWM(MOTOR_PWM, 0);           // from jerking forward when the program starts.
gpioServo(SERVO_PWM, 2150);

// -----
// PARAMETERS
// -----

int  motorSpeed  = 85;
double Kp        = 0.6;
double Ki        = 0.005;
int  startDelaySec = 3;
double smoothing  = 0.35;
int  minArea     = 50;
double maxAreaRatio = 2.5;
double targetFPS  = 5.0;    // <-- change to 5.0-30 for next tests
double targetFrameTime = 1.0 / targetFPS;

// -----

// Start camera via GStreamer

string pipeline = "libcamerasrc ! video/x-raw,width=640,height=480,framerate=30/1 ! videoconvert ! video/x-raw,format=BGR ! appsink drop=true sync=false";
VideoCapture cap(pipeline, CAP_GSTREAMER);

if (!cap.isOpened()) {
    cerr << "Could not open camera!" << endl;
    safeShutdown();
    return -1;
}

// Create video file for logging the test run
time_t now_t = time(nullptr);
char timestr[64];
strftime(timestr, sizeof(timestr), "%Y%m%d_%H%M%S", localtime(&now_t));
string videoFile = "/home/hmm/conebot_run_" + string(timestr) + ".avi";

VideoWriter writer;

```

```

writer.open(videoFile,
            VideoWriter::fourcc('M','J','P','G'),
            targetFPS,
            Size(640, 480));

if (!writer.isOpened()) {
    cerr << "Could not create video file: " << videoFile << endl;
    cap.release();
    safeShutdown();
    return -1;
}

cout << "=== CONEBOT V5 ===" << endl;
cout << "Recording to: " << videoFile << endl;
cout << "Kp=" << Kp << " Ki=" << Ki
    << " Smoothing=" << smoothing
    << " Speed=" << motorSpeed
    << " FPS=" << targetFPS << endl;
cout << "Starting in " << startDelaySec << " seconds..." << endl;

// Initialize variables for steering, timing and logging
Mat frame, hsv, maskBlue, maskYellow;
int  currentSteering = 2150;
double smoothedSteering = 2150.0;
double integral      = 0.0;
auto  prevTime      = chrono::high_resolution_clock::now();
auto  startTime     = chrono::high_resolution_clock::now();
bool  motorStarted  = false;
int   frameCount    = 0;

while (keepRunning) {
    // Read next frame and start FPS timing
    // FPS limiting: start timing for this frame
    auto fpsStart = chrono::high_resolution_clock::now();

    cap >> frame;
    if (frame.empty()) continue;

```

```

flip(frame, frame, -1);

frameCount++;

// Convert image to HSV and create color masks
cvtColor(frame, hsv, COLOR_BGR2HSV);

inRange(hsv, Scalar(109, 129, 23), Scalar(130, 255, 255), maskBlue);
inRange(hsv, Scalar(17, 175, 142), Scalar(34, 255, 255), maskYellow);

// Find contours for blue and yellow cones
vector<vector<Point>> contoursBlue, contoursYellow;

findContours(maskBlue, contoursBlue, RETR_EXTERNAL, CHAIN_APPROX_SIMPLE);
findContours(maskYellow, contoursYellow, RETR_EXTERNAL, CHAIN_APPROX_SIMPLE);

// Find largest yellow (reference)
double maxAreaYellow = 0;
Rect bestYellow;
for (const auto& c : contoursYellow) {
    double a = contourArea(c);
    if (a > maxAreaYellow) {
        maxAreaYellow = a;
        bestYellow = boundingRect(c);
    }
}

// Find blue cone that best matches yellow in area
double maxAreaBlue = 0;
double bestBlueDiff = 1e9;
Rect bestBlue;
for (const auto& c : contoursBlue) {
    double a = contourArea(c);
    if (a < minArea) continue;
    if (maxAreaYellow > 0) {
        double diff = abs(a - maxAreaYellow);
        if (diff < bestBlueDiff) {
            bestBlueDiff = diff;

```

```

        maxAreaBlue = a;

        bestBlue = boundingRect(c);
    }
} else {
    if (a > maxAreaBlue) {
        maxAreaBlue = a;
        bestBlue = boundingRect(c);
    }
}
}

// Visualization
Mat display = frame.clone();
line(display, Point(320, 0), Point(320, 480), Scalar(255,255,255), 1);

// All contours as thin boxes
for (const auto& c : contoursBlue)
    if (contourArea(c) > 30)
        rectangle(display, boundingRect(c), Scalar(150,80,0), 1);
for (const auto& c : contoursYellow)
    if (contourArea(c) > 30)
        rectangle(display, boundingRect(c), Scalar(0,100,150), 1);

// Best cones in thick lines
if (maxAreaBlue > minArea) {
    rectangle(display, bestBlue, Scalar(255,0,0), 3);
    circle(display,
        Point(bestBlue.x + bestBlue.width/2,
            bestBlue.y + bestBlue.height/2),
        6, Scalar(255,0,0), -1);
    putText(display, "BLUE " + to_string((int)maxAreaBlue),
        Point(bestBlue.x, max(bestBlue.y-8, 15)),
        FONT_HERSHEY_SIMPLEX, 0.5, Scalar(255,0,0), 2);
}
if (maxAreaYellow > minArea) {
    rectangle(display, bestYellow, Scalar(0,200,255), 3);
    circle(display,

```

```

        Point(bestYellow.x + bestYellow.width/2,
              bestYellow.y + bestYellow.height/2),
        6, Scalar(0,200,255), -1);

putText(display, "YELLOW " + to_string((int)maxAreaYellow),
        Point(bestYellow.x, max(bestYellow.y-8, 15)),
        FONT_HERSHEY_SIMPLEX, 0.5, Scalar(0,200,255), 2);
}

if (maxAreaBlue > minArea && maxAreaYellow > minArea) {

    double areaRatio = maxAreaBlue / maxAreaYellow;

    if (areaRatio > maxAreaRatio || areaRatio < (1.0 / maxAreaRatio)) {

        // Wrong pair
        integral = 0.0;

        prevTime = chrono::high_resolution_clock::now();

        smoothedSteering = smoothing * smoothedSteering
            + (1.0 - smoothing) * 2150.0;

        putText(display, "WRONG PAIR ratio:" +
            to_string(areaRatio).substr(0,4),
            Point(10,30), FONT_HERSHEY_SIMPLEX,
            0.7, Scalar(0,100,255), 2);

        cout << "WRONG PAIR ratio=" << areaRatio
            << " Blue:" << (int)maxAreaBlue
            << " Yellow:" << (int)maxAreaYellow << endl;

    } else {

        // Correct pair

        double blueCX  = bestBlue.x  + bestBlue.width  / 2.0;
        double yellowCX = bestYellow.x + bestYellow.width / 2.0;

        double lookAheadX = (blueCX + yellowCX) / 2.0;

        double blueCY  = bestBlue.y  + bestBlue.height  / 2.0;
        double yellowCY = bestYellow.y + bestYellow.height / 2.0;

        double lookAheadY = (blueCY + yellowCY) / 2.0;

```

```

double cte = 320.0 - lookAheadX;

auto now2 = chrono::high_resolution_clock::now();

double dt = chrono::duration<double>(now2 - prevTime).count();

prevTime = now2;

integral += cte * dt;

double output = (Kp * cte) + (Ki * integral);

currentSteering = 2150 - static_cast<int>(output);

currentSteering = max(1900, min(2400, currentSteering));

smoothedSteering = smoothing * smoothedSteering
    + (1.0 - smoothing) * currentSteering;

line(display,
    Point((int)blueCX, (int)blueCY),
    Point((int)yellowCX, (int)yellowCY),
    Scalar(0,255,0), 2);

circle(display,
    Point((int)lookAheadX, (int)lookAheadY),
    10, Scalar(0,255,0), -1);

line(display, Point(320,480),
    Point((int)lookAheadX, (int)lookAheadY),
    Scalar(0,255,0), 2);

putText(display, "CTE: " + to_string((int)cte),
    Point(10,30), FONT_HERSHEY_SIMPLEX, 0.7, Scalar(0,255,0), 2);

putText(display, "PWM: " + to_string((int)smoothedSteering),
    Point(10,60), FONT_HERSHEY_SIMPLEX, 0.7, Scalar(0,255,0), 2);

putText(display, "Ratio: " + to_string(areaRatio).substr(0,4),
    Point(10,90), FONT_HERSHEY_SIMPLEX, 0.6, Scalar(200,255,0), 1);

cout << "CTE: " << (int)cte
    << " | PWM: " << (int)smoothedSteering
    << " | Ratio: " << to_string(areaRatio).substr(0,4) << endl;
}

```

```

} else {

    integral = 0.0;

    prevTime = chrono::high_resolution_clock::now();

    smoothedSteering = smoothing * smoothedSteering
        + (1.0 - smoothing) * 2150.0;

    string missing = "";

    if (maxAreaBlue <= minArea) missing += "MISSING BLUE ";

    if (maxAreaYellow <= minArea) missing += "MISSING YELLOW";

    putText(display, missing,
        Point(10,30), FONT_HERSHEY_SIMPLEX, 0.8, Scalar(0,0,255), 2);

    putText(display, "Blue:" + to_string((int)maxAreaBlue) +
        " Yellow:" + to_string((int)maxAreaYellow),
        Point(10,60), FONT_HERSHEY_SIMPLEX, 0.6, Scalar(0,100,255), 2);

    cout << missing
        << " Blue:" << (int)maxAreaBlue
        << " Yellow:" << (int)maxAreaYellow << endl;
}

// Steering indicator

int barCX = 320, barY = 455, barHW = 180;

rectangle(display, Point(barCX-barHW, barY-7),
    Point(barCX+barHW, barY+7), Scalar(50,50,50), -1);

line(display, Point(barCX, barY-11),
    Point(barCX, barY+11), Scalar(255,255,255), 1);

int barOff = (int)((((smoothedSteering-2150)/250.0)*barHW);

rectangle(display, Point(barCX, barY-5),
    Point(barCX+barOff, barY+5),
    barOff > 0 ? Scalar(0,150,255) : Scalar(255,150,0), -1);

// FPS display on screen

putText(display, "FPS: " + to_string((int)targetFPS),
    Point(10, 150), FONT_HERSHEY_SIMPLEX, 0.6, Scalar(200,200,0), 1);

// Motor start

```

```

double elapsed = chrono::duration<double>(
    chrono::high_resolution_clock::now() - startTime).count();

if (!motorStarted && elapsed >= startDelaySec) {
    cout << ">>> MOTOR STARTING <<<" << endl;
    gpioWrite(MOTOR_HIGH, 1);
    gpioWrite(MOTOR_LOW, 0);
    gpioPWM(MOTOR_PWM, motorSpeed);
    motorStarted = true;
}

if (!motorStarted) {
    putText(display,
        "Start in: " + to_string(startDelaySec-(int)elapsed) + "s",
        Point(10,120), FONT_HERSHEY_SIMPLEX, 0.8, Scalar(0,255,255), 2);
} else {
    putText(display, "MOTOR ON",
        Point(10,120), FONT_HERSHEY_SIMPLEX, 0.6, Scalar(0,255,0), 1);
}

gpioServo(SERVO_PWM, (int)smoothedSteering);

if (writer.isOpened()) {
    writer.write(display);
}

// ——— FPS LIMITING ———
auto fpsEnd = chrono::high_resolution_clock::now();
double frameElapsed = chrono::duration<double>(fpsEnd - fpsStart).count();
if (frameElapsed < targetFrameTime) {
    gpioDelay((unsigned)((targetFrameTime - frameElapsed) * 1000000));
}

// ———
}

safeShutdown();
cap.release();

```

```
if (writer.isOpened()) {  
    writer.release();  
}  
  
cout << "Video saved: " << videoFile << endl;  
cout << "Frames recorded: " << frameCount << endl;  
return 0;  
}
```

Appendix B: MATLAB

```
% ConeBot graphs - FPS vs speed and CTE analysis
```

```
% MF133x KTH
```

```
% Video files:
```

```
% 30 FPS: conebot_run_20260428_170207.avi (110 PWM)
```

```
% 15 FPS: conebot_run_20260428_202626.avi (95 PWM)
```

```
% 10 FPS: conebot_run_20260428_203840.avi (90 PWM)
```

```
% 5 FPS: conebot_run_20260428_211039.avi (85 PWM)
```

```
clear; clc; close all;
```

```
% Test data
```

```
fps = [5, 10, 15, 30];
```

```
pwm = [85, 90, 95, 110];
```

```
times = [16.8, 14.16, 9.0, 6.0];
```

```
speed = 4.8 ./ times;
```

```
% CTE values per FPS (frames marked as WRONG PAIR / MISSING removed)
```

```
cte5 = [5,6,3,2,0,-4,-13,-22,-35,-45,-53,-68,-78,-99,-103,-106,-164,-160,...
```

```
-152,-122,6,64,58,50,38,10,-21,-60,-88,-127,-142,-143,-136,-110,...
```

```
-91,-67,-41,-9,13,26,29,22,16,-5,-20,-53,-92,-110,-127,-143,-134,...
```

```
-123,-101,-83,-48,-21,6,21,29,26,14,-2,-99,-46,-70,-83,-87,-84,...
```

```
-78,-74,-66,-54,-43,-33,-20,-9,-6,-7];
```

```
cte10 = [13,13,13,13,13,8,9,3,2,1,-2,-9,-16,-21,-26,-31,-38,-41,-47,-53,...
```

```
-61,-70,-77,-79,-80,-99,-101,-104,-105,-155,-153,-146,-145,-141,...
```

```
-142,-135,-120,-5,9,-41,-44,65,60,53,46,46,45,31,20,14,-8,-24,...
```

```
-50,-67,-80,-170,-143,-151,-153,-156,-203,-194,-181,-94,-72,-47,...
```

```
14,19,29,35,42,45,38,32,23,17,7,-7,-24,-44,-75,-91,-150,-154,...
```

```
-127,-133,-137,-134,-125,-100,-82,-66,-55,-32,-11,8,23,35,39,45,...
```

```
43,38,28,15,6,-2,-17,-44,-57,-67,-143,-121,-130,-140,-148,-157,...
```

```
-149,-144,-133,-124,-111,-92,-70,-48,-17,7,25,42,52,62,65,66,56,...
```

```
46,27,2,-8,-16];
```

```
cte15 = [12,12,12,13,12,12,11,8,4,1,-6,-9,-13,-19,-24,-28,-39,-48,-52,...
-68,-72,-81,-83,-91,-95,-111,-116,-115,-165,-162,-160,-157,-105,...
-97,-87,-132,-121,-102,-7,8,-24,103,101,99,96,92,88,84,74,53,40,...
13,-16,-49,-76,-161,-148,-176,-199,-210,-206,-195,-185,-160,-136,...
-118,-116,-104,-76,-57,-16,-6,35,60,74,91,95,104,103,100,87,81,...
63,35,14,-15,-26,-126,-97,-124,-150,-163,-173,-177,-177,-176,...
-168,-158,-145,-123,-116,-82,-54,-35,-3,9,53,64,85,88,92,91,89,...
52,39,11,-1,-115,-97,-122,-129,-143,-148,2,-141,-127,-120];
```

```
cte30 = [5,5,5,5,6,7,6,4,0,0,-4,-10,-11,-15,-17,-24,-29,-33,-39,...
-44,-47,-51,-55,-57,-61,-66,-74,-75,-77,-76,-71,-83,-83,-85,-80,...
-76,-67,-130,-124,-120,-115,-3,1,-3,-6,-5,2,16,31,59,65,64,64,63,...
61,61,59,48,37,29,5,-20,-35,-56,-66,-146,-117,-129,-140,-154,-159,...
-164,-166,-157,-148,-134,-123,-94,-75,-56,-11,10,35,55,125,139,-46,...
24,13,1,-5,-8,-18,-31,-37,-40,-42,-41,-41,-153,-148,-156,-168,-170,...
-169,-172,-175,-173,-169,-146,-126,-106,-63,-40,-16,6,60,112];
```

```
cte_all = {cte5, cte10, cte15, cte30};
```

```
% Colors per FPS
```

```
colors = [0.8 0.2 0.2;
```

```
0.9 0.6 0.1;
```

```
0.2 0.6 0.9;
```

```
0.1 0.7 0.3];
```

```
% Output folder
```

```
outdir = 'ConeBot_English_figures';
```

```
if ~exist(outdir, 'dir')
```

```
    mkdir(outdir);
```

```
end
```

```
%% Figure 4-1 - FPS vs maximum stable speed
```

```
figure('Units','centimeters','Position',[2 2 16 10]);

plot(fps, speed, '-o', 'LineWidth', 2, 'MarkerSize', 8, ...

    'Color', [0.15 0.45 0.75], 'MarkerFaceColor', [0.15 0.45 0.75]);

for i = 1:4

    text(fps(i)+0.4, speed(i)+0.01, sprintf('%2f m/s', speed(i)), 'FontSize', 10);

end

xlabel('Frame rate (FPS)');

ylabel('Speed (m/s)');

title('Frame rate vs maximum stable speed');

xlim([3 33]); ylim([0 1.0]);

xticks(fps);

grid on;

exportgraphics(gcf, fullfile(outdir, 'Figure_4_1_frame_rate_vs_speed_EN.png'), 'Resolution', 300);

exportgraphics(gcf, fullfile(outdir, 'Figure_4_1_frame_rate_vs_speed_EN.pdf'), 'ContentType', 'vector');
```

```
%% Figure 4-2 - FPS vs maximum stable PWM
```

```
figure('Units','centimeters','Position',[2 2 16 10]);

plot(fps, pwm, '-s', 'LineWidth', 2, 'MarkerSize', 8, ...

    'Color', [0.75 0.3 0.1], 'MarkerFaceColor', [0.75 0.3 0.1]);

for i = 1:4

    text(fps(i)+0.4, pwm(i)+0.6, sprintf('%d PWM', pwm(i)), 'FontSize', 10);

end

xlabel('Frame rate (FPS)');

ylabel('PWM');

title('Frame rate vs maximum stable PWM');

xlim([3 33]); ylim([80 115]);

xticks(fps);

grid on;
```

```
%% Figure 4-3 - CTE over time
```

```
order = [4 3 2 1]; % 30, 15, 10, 5 FPS

figure('Units','centimeters','Position',[2 2 18 13]);
```

```

for k = 1:4

    i = order(k);

    dt = 1/fps(i);

    t = (0:length(cte_all{i})-1) * dt;

    subplot(2,2,k);

    area(t, cte_all{i}, 'FaceColor', colors(i,:), 'FaceAlpha', 0.25, ...

        'EdgeColor', colors(i,:), 'LineWidth', 1.2);

    hold on;

    plot(t, zeros(size(t)), 'k--', 'LineWidth', 1);

    xlabel('Time (s)');

    ylabel('CTE (px)');

    title(sprintf('%d FPS', fps(i)));

    ylim([-280 280]);

    grid on;

end

sgtitle('Cross-Track Error (CTE) over time per frame rate');

%% Figure 4-4 - CTE standard deviation

cte_std = cellfun(@std, cte_all);

figure('Units','centimeters','Position',[2 2 16 10]);

b = bar(fps, cte_std, 0.5);

b.FaceColor = 'flat';

for i = 1:4

    b.CData(i,:) = colors(i,:);

end

for i = 1:4

    text(fps(i), cte_std(i)+1.5, sprintf('%0.1f', cte_std(i)), ...

        'HorizontalAlignment', 'center', 'FontSize', 10);

end

xlabel('Frame rate (FPS)');

ylabel('CTE standard deviation (px)');

```

```

title('CTE stability per frame rate (lower is better)');

xticks(fps);

grid on;

%% Figure 4-5 - CTE histogram

figure('Units','centimeters','Position',[2 2 20 8]);

for k = 1:4

    i = order(k);

    subplot(1,4,k);

    histogram(cte_all{i}, 20, 'FaceColor', colors(i,:), ...

        'EdgeColor', 'white', 'FaceAlpha', 0.85);

    xlabel('CTE (px)');

    ylabel('Count');

    title(sprintf('%d FPS', fps(i)));

    xlim([-280 280]);

    grid on;

end

sgtitle('Distribution of CTE values per frame rate');

%% Printed table

fprintf('\nFPS | PWM | Time(s) | Speed(m/s) | CTE std\n');

fprintf('----+----+-----+-----+-----\n');

for i = 1:4

    fprintf('%3d | %3d | %7.2f | %10.3f | %7.1f\n', ...

        fps(i), pwm(i), times(i), speed(i), cte_std(i));

end

```

Appendix C: Video Clips

<https://github.com/hello-suprice/ConeBot/tree/main/FPS>

FPS	Video file
30	conebot_run_20260428_170207.avi
15	conebot_run_20260428_202626.avi
10	conebot_run_20260428_203840.avi
5	conebot_run_20260428_211039.avi

